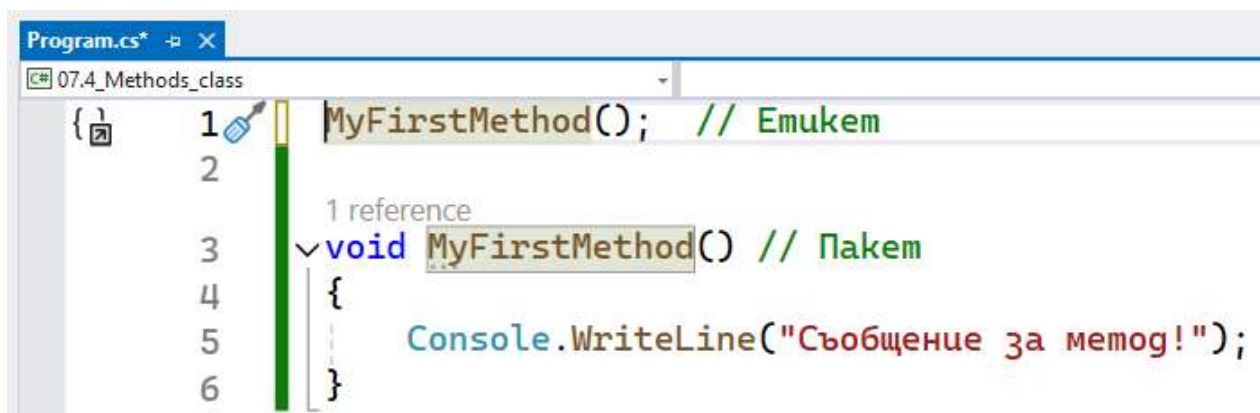


МЕТОДИ – ВИДОВЕ

Предния час казахме, че можем да разглеждаме кода на методите като математически формули, които да извикваме по име и им задаваме различни стойности, които да обработват. Формулата (кодът) е отделен, а като извикваме името, между скобите му подаваме различни стойности.

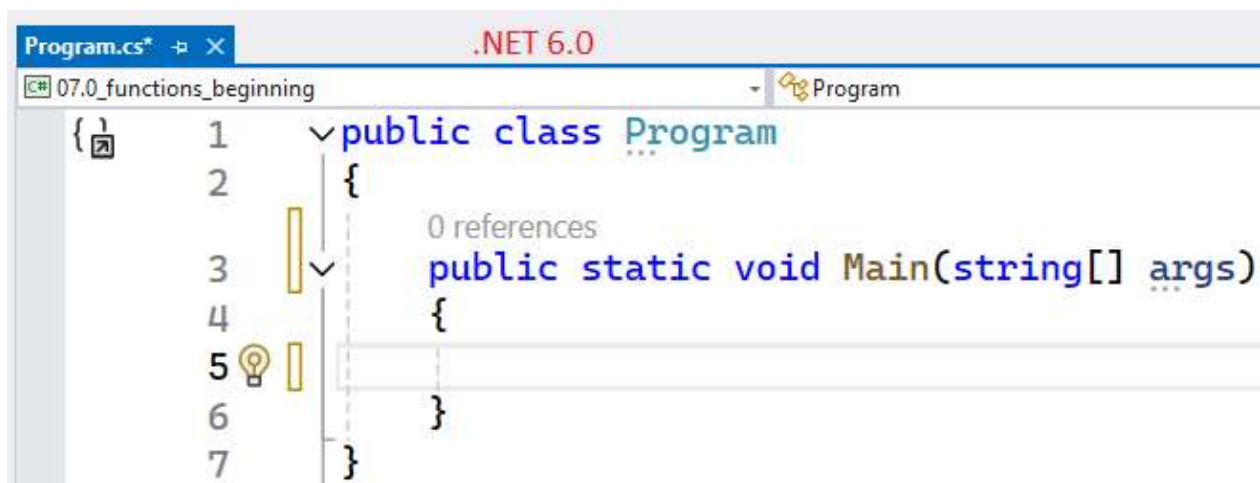
Кодът на `Sum(int a, int b){}` е отделен, той пази формулата, а ние можем да подаваме различни стойности като извикваме името `Sum(2,3); Sum(5,10)` и т.н.

Също можем да асоциираме клас `Program.cs` с книга – в `Main()`-метода е съдържанието – коя глава след коя следва и на кои страници можем да ги прочетем подробно. Отделната част с повече текст е кодът на съответните методи (дефиницията им).



```
Program.cs*  X
07.4_Methods_class
1  MyFirstMethod(); // Emukem
2
3  void MyFirstMethod() // Пакем
4  {
5      Console.WriteLine("Съобщение за метод!");
6  }
```

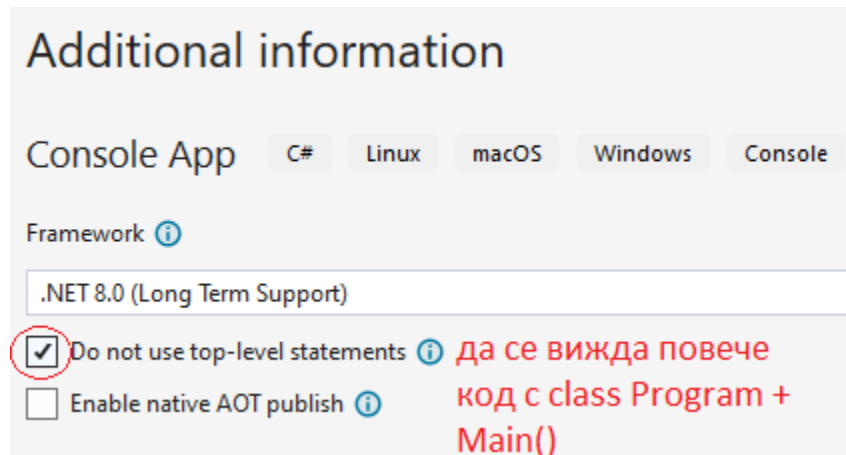
И видяхме, че в по-старата версия .Net 6.0 се вижда малко повече код.



```
Program.cs*  X .NET 6.0
07.0_functions_beginning Program
1  public class Program
2  {
3      public static void Main(string[] args)
4      {
5      }
6  }
```

```
public class Program
{
    public static void Main(string[] args)
    {
    }
}
```

За да пишем всички с еднаква видимост на кода, най-добре е при създаването на нов проект да слагаме отметка “Do not use top-level statements”:



Additional information

Console App C# Linux macOS Windows Console

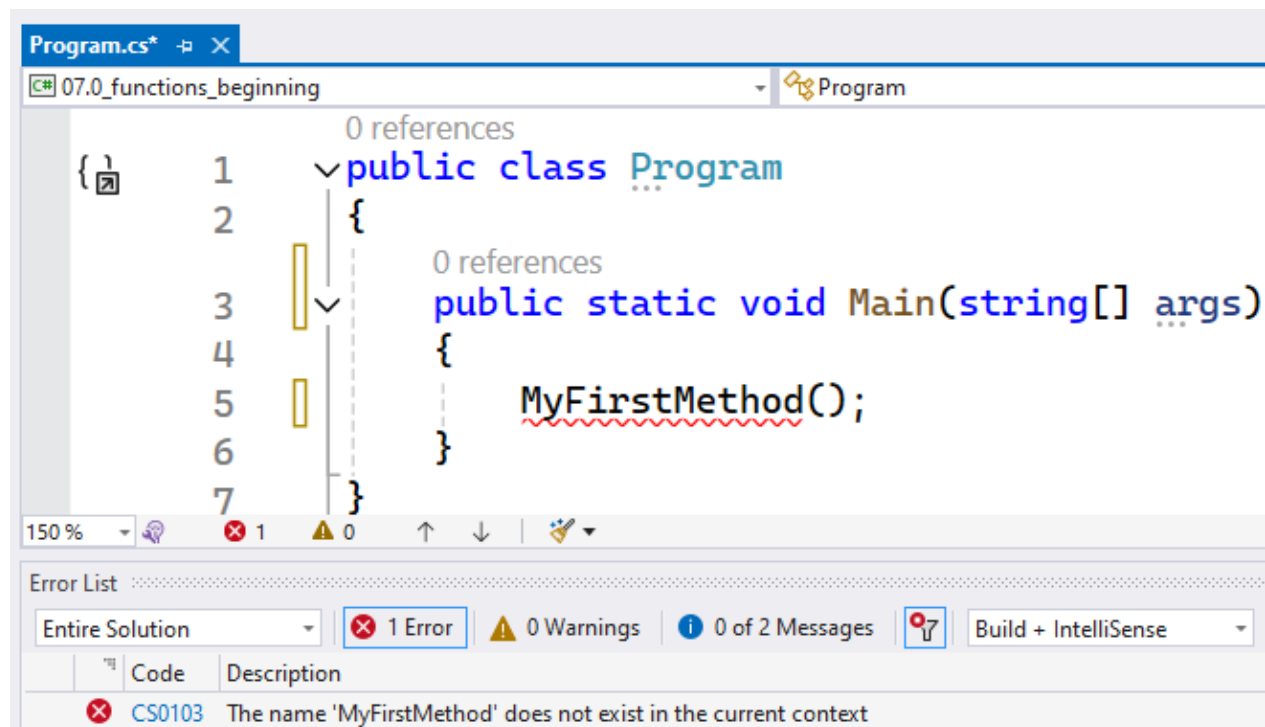
Framework ⓘ

.NET 8.0 (Long Term Support)

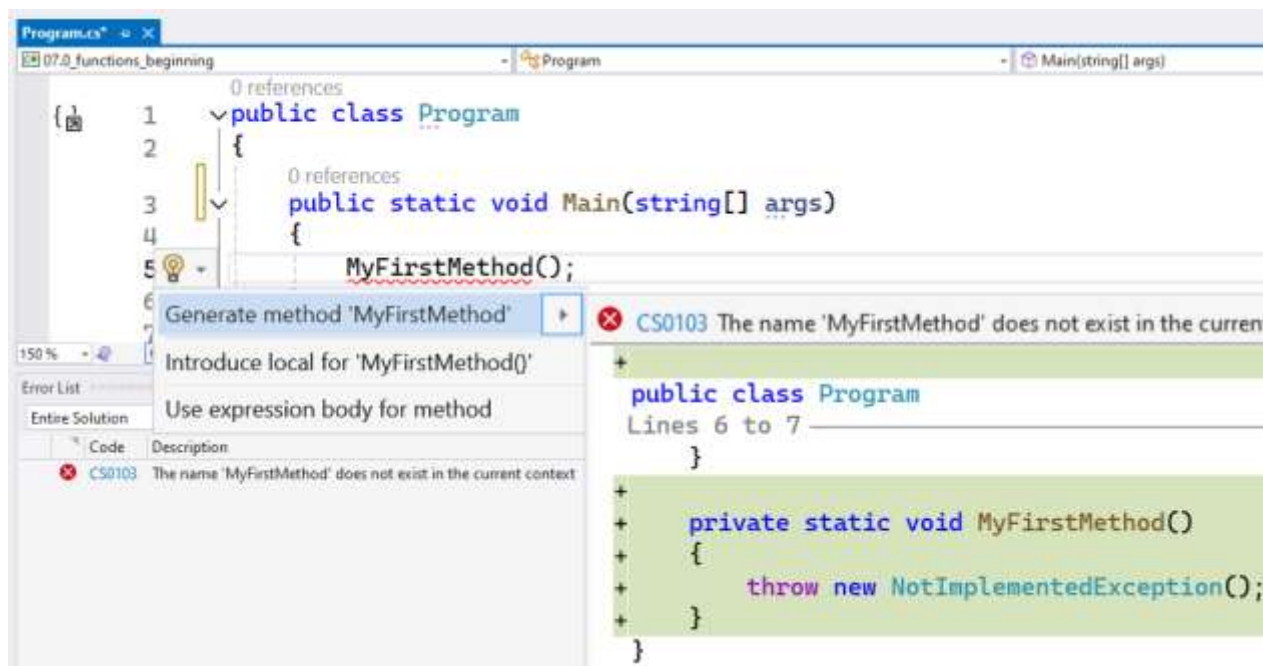
☒ Do not use top-level statements ⓘ да се вижда повече код с class Program + Main()

☐ Enable native AOT publish ⓘ

Досега писахме код, който „опаковахме“ като метод и го извиквахме един или няколко пъти с подаване на различни стойности за променливите. Нека видим и обратния начин: може първо да извикаме името на даден метод, който ни трябва, а после да напишем кода му.



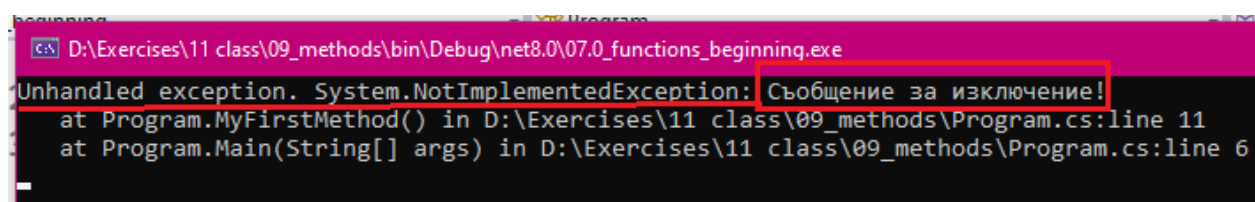
Метод MyFirstMethod(); още не съществува, но можем да го генерираме автоматично – ако натиснем **Ctrl + .** за автокорекция



Генерира се частен (private) метод. Когато методът е public, той е достъпен от всяка част, но при private имаме защита, която ще разгледаме малко по-нататък. Void за нас означаваше, че ще се изведе съобщение на конзолата Console.WriteLine, но вслучая при автогенериране имаме хвърляне на изключение (throw new Exception), което обикновено означава прихващане на грешка. То обикновено се състои също в показване на съобщение на конзолата:

```
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        MyFirstMethod();
    }

    1 reference
    private static void MyFirstMethod()
    {
        throw new NotImplementedException("Съобщение за изключение!");
    }
}
```



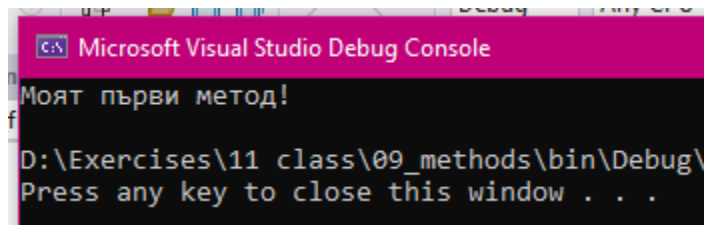
Много лесно може да го редактираме до познатия начин:

```

public class Program
{
    0 references
    public static void Main(string[] args)
    {
        MyFirstMethod();
    }

    1 reference
    private static void MyFirstMethod()
    {
        Console.WriteLine("Моят първи метод!");
    }
}

```

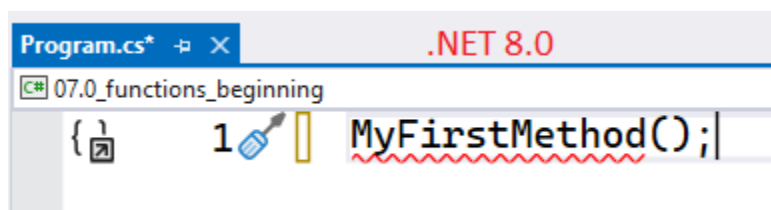


```

Microsoft Visual Studio Debug Console
Моят първи метод!
D:\Exercises\11 class\09_methods\bin\Debug\
Press any key to close this window . . .

```

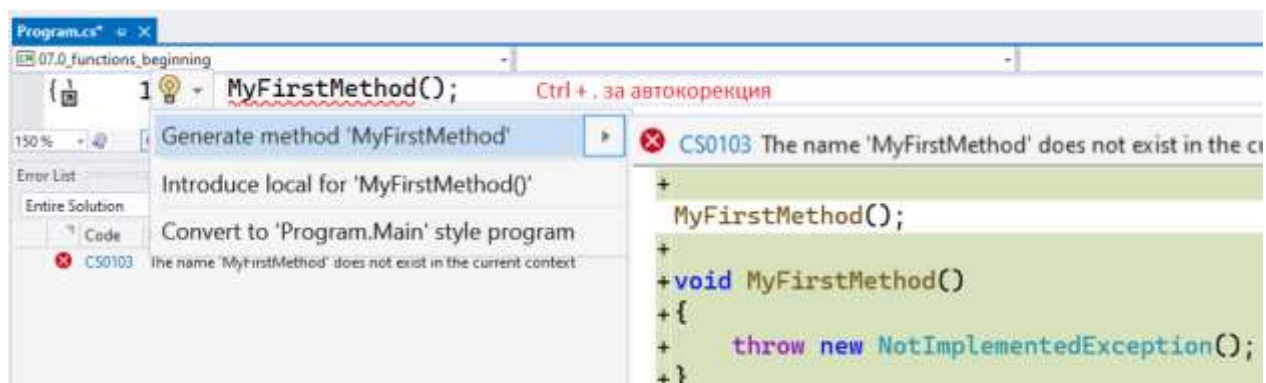
Ако сме във версия 8.0, припомням, че class Program + Main() са скрити и пишем направо:



```

Program.cs* .NET 8.0
07.0_functions_beginning
{
    1 MyFirstMethod();
}

```



```

Program.cs*
07.0_functions_beginning
{
    1 MyFirstMethod();
}

```

Generate method 'MyFirstMethod'

Introduce local for 'MyFirstMethod()'

Convert to 'Program.Main' style program

CS0103 The name 'MyFirstMethod' does not exist in the current context

```

+ MyFirstMethod();
+ void MyFirstMethod()
+ {
+     throw new NotImplementedException();
+ }

```

Съответно отново се генерира метод, който просто може да изведе съобщение.

```
Program.cs* X
07.0_functions_beginning
1
2 MyFirstMethod();
3
4 1 reference
5 void MyFirstMethod()
6 {
7     throw new NotImplementedException();
8 }
```

Който също може да се промени на Console.WriteLine както вече показахме.

```
GreetUser("Иван");
GreetUser("Димитър");
string name = Console.ReadLine(); // Контейнер name
GreetUser(name);

3 references
void GreetUser(string userName) // Математическа формула
{
    Console.WriteLine($"Здравей, {userName}");
}
```

СТОЙНОСТИ

userName важи само за формулата (кода), докато какво му се подава (name като променлива или директна стойност) – може да се нарича по всякакъв начин. Подадената стойност (Input variable) ще бъде сложена на мястото на userName и обработена както сме задали във формулата.

Друг пример:

```
int result = Substraction(5, 8);
int c = 10;
Substraction(result, c);
Console.WriteLine(Substraction(15,10));
```

резултат от метод
в друг метод

метод в друг метод



GreetUser("Иван");

Generate method 'GreetUser'

Use local for 'GreetUser("Иван")'

Port to 'Program.Main' style program

```

string name = Console.ReadLine();
GreetUser(name);

```

```

/*
void GreetUser(string userName)
{

```

CS0103 The name 'GreetUser' does not exist in the current context
Lines 2 to 3

Separator();

разпозна го като тип string

void GreetUser(string v)

throw new NotImplementedException();

```

14 void GreetUser(string userName)
15 {
16     Console.WriteLine($"3gpa");
17 }
18
19 Separator();
20
21 Substraction(8, 5);

```

Generate method 'Substraction'

Lines 22 to 23

автоматично ги разпозна като тип int и предлага

void Substraction(int v1, int v2)

throw new NotImplementedException();

void MyFirstMethod() // Пакем

Preview changes

1. Локални функции (вътре в Program.cs) – Top-level code

```
Program.cs X
C# 07.0_functions_beginning

1 // Top-level code
2 Console.WriteLine("=== Локални функции ===");
3 Console.Write("Въведете име: ");
4 string name = Console.ReadLine();
5 GreetUser(name);
6 Subtraction(8, 5);
7 Console.WriteLine($"Факториел от 5: {Factorial(5)}");
8
9 void GreetUser(string userName)
10 {
11     Console.WriteLine($"Здравей, {userName}!");
12 }
13
14 int Subtraction(int a, int b)
15 {
16     int difference = a - b;
17     Console.WriteLine($"{a} - {b} = {difference}");
18     return difference;
19 }
20
21 int Factorial(int n) // Рекурсия - извиква сама себе с
22 {
23     if (n <= 1) return 1;
24     return n * Factorial(n - 1);
25 }
```

СТОЙНОСТИ
СТОЙНОСТИ
СТОЙНОСТИ

формула

формула

формула

Рекурсия – функция, която извиква сама себе си, т.е. в тялото на самия метод трябва да стои името на същия метод. Рекурсиите са много подмолни, т.к. могат да доведат до безкраен цикъл.

```
Microsoft Visual Studio Debug Console

=== Локални функции ===
Въведете име: Николета
Здравей, Николета!
8 - 5 = 3
Факториел от 5: 120

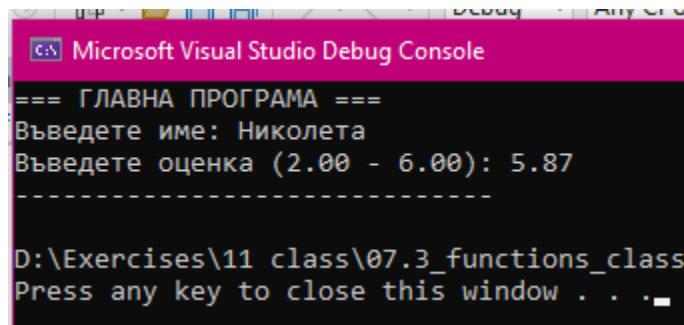
D:\Exercises\11 class\09_methods\bin\Debug
Press any key to close this window . . .
```

2. Отделни класове (препоръчително)



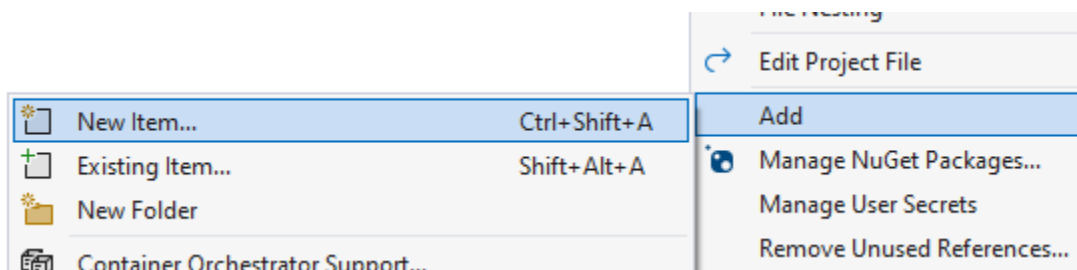
```
1 // Top-level code
2 using System;
3
4 Console.WriteLine("=== ГЛАВНА ПРОГРАМА ===");
5 Console.Write("Въведете име: ");
6 string username = Console.ReadLine();
7 Console.Write("Въведете оценка (2.00 - 6.00): ");
8 double grade = double.Parse(Console.ReadLine());
9 DisplaySeparator(); // локална функция
10
11 void DisplaySeparator()
12 {
13     Console.WriteLine(new string('-', 30));
14 }
```

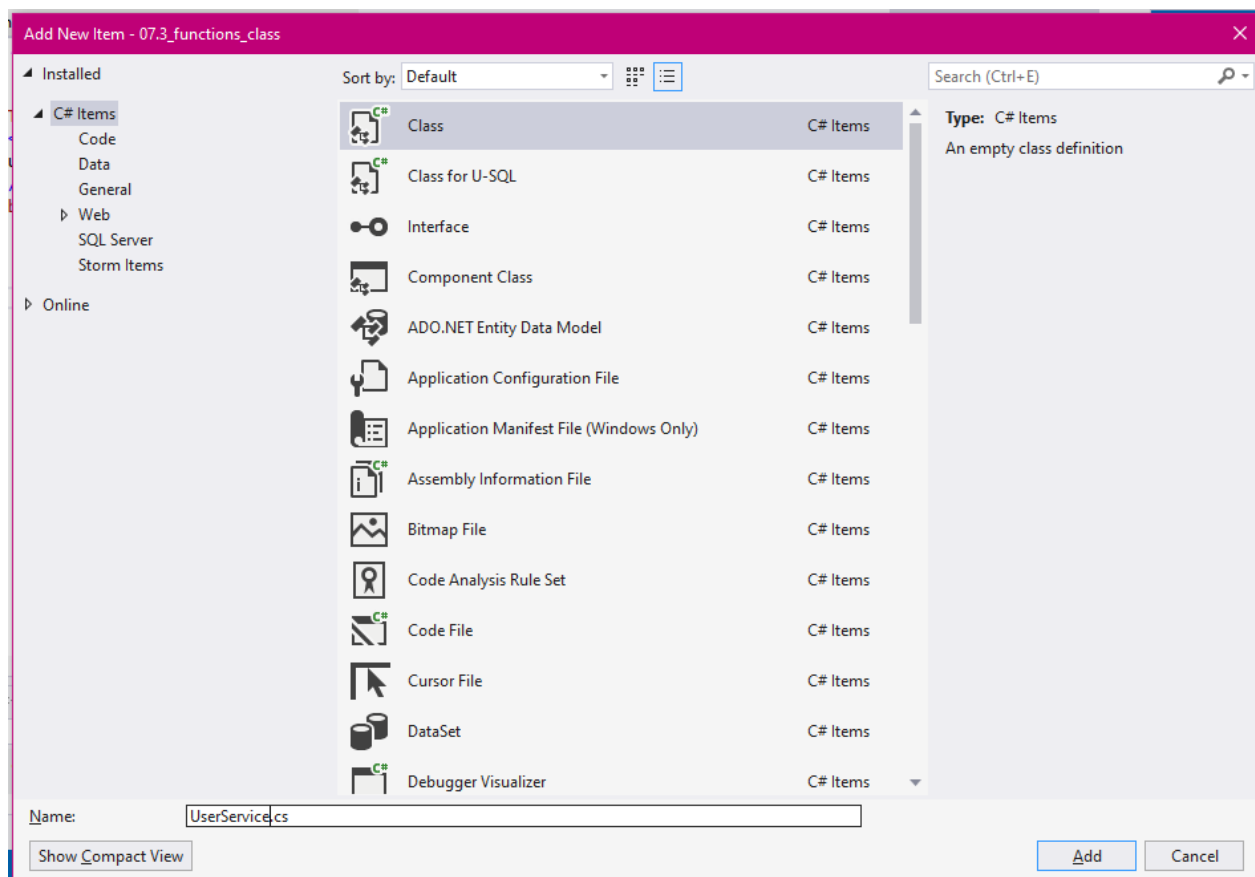
Дотук всичко е познато, освен изписването на разделител. Виждаме обаче 3 съседни файла, които трябва да създадем. Това са класове .cs (C sharp). Главният файл е Program.cs, но той може да извиква класове от други файлове, а те да имат в себе си по няколко метода, които също могат да се извикват чрез наследяване точка .



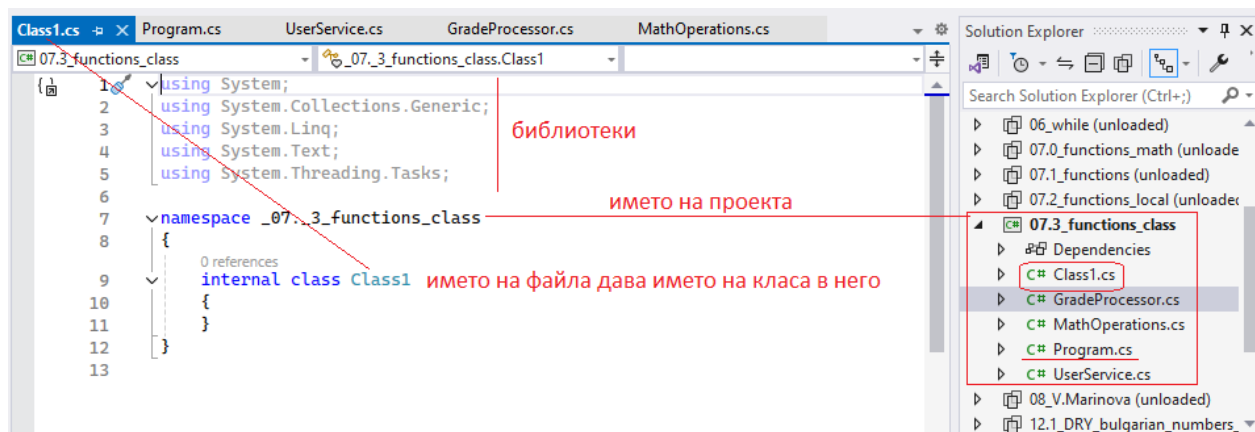
```
=== ГЛАВНА ПРОГРАМА ===
Въведете име: Николета
Въведете оценка (2.00 - 6.00): 5.87
-----
D:\Exercises\11 class\07.3_functions_class
Press any key to close this window . . .
```

Нека ги създадем като маркираме името на проекта -> д. б. -> Add -> New Item





Другият начин е директно -> Add -> Class

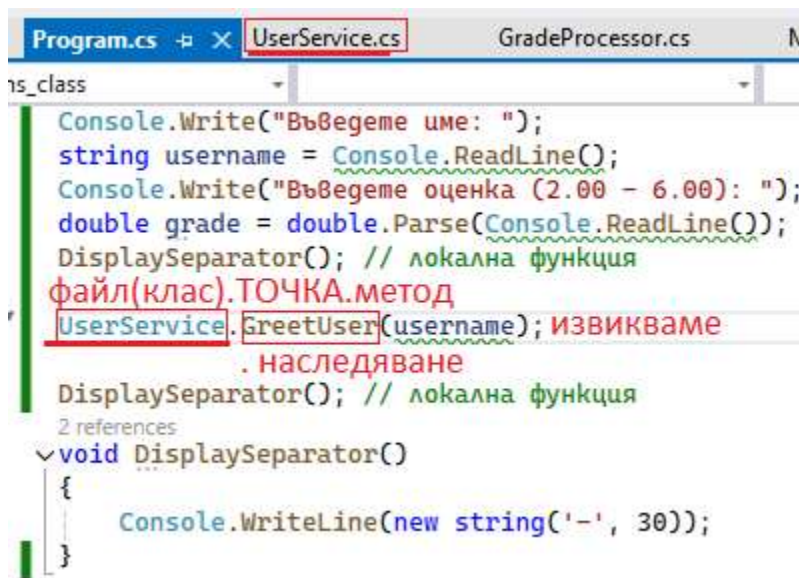


Нека разгледаме автогенерирания код на празен клас Class1.cs. След като забележахме зависимостите, да си създадем 3 файла за класове, които ще използваме. Нека започнем от UserService.cs:



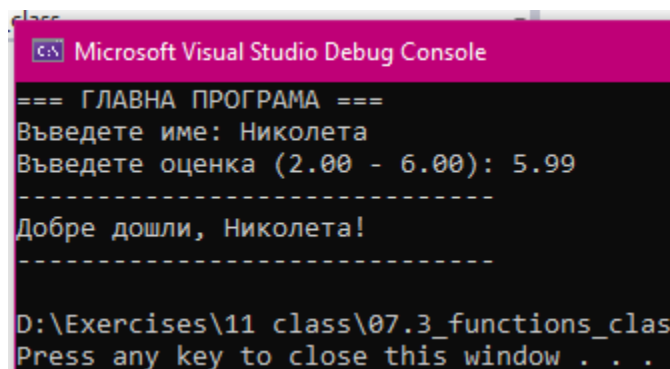
```
Program.cs | UserService.cs | GradeProcessor.cs | Mat
15_class
  ▼ public static class UserService
  {
    0 references
    public static void GreetUser(string name)
    {
      Console.WriteLine($"Добре дошли, {name}!");
    }
  }
```

Този клас е публичен, т.е. общодостъпен без ограничения и има само 1 метод в него, който изпечатва на конзолата поздрав. Всеки клас може да има по много методи, които могат да бъдат извиквани чрез наследяване с точка. Нека видим.



```
Program.cs | UserService.cs | GradeProcessor.cs | M
15_class
  Console.WriteLine("Въведете име: ");
  string username = Console.ReadLine();
  Console.WriteLine("Въведете оценка (2.00 - 6.00): ");
  double grade = double.Parse(Console.ReadLine());
  DisplaySeparator(); // локална функция
  файл(клас).ТОЧКА.метод
  UserService.GreetUser(username); извикваме
  .наследяване
  DisplaySeparator(); // локална функция
  2 references
  void DisplaySeparator()
  {
    Console.WriteLine(new string('-', 30));
  }
```

След като нашият файл UserService.cs е готов и в него има метод GreetUser(), той бива разпознат от IntelliSense и може да бъде използван от Top-level файла Program.cs:



```
Microsoft Visual Studio Debug Console
=== ГЛАВНА ПРОГРАМА ===
Въведете име: Николета
Въведете оценка (2.00 - 6.00): 5.99
-----
Добре дошли, Николета!
-----
D:\Exercises\11 class\07.3_functions_class
Press any key to close this window . . .
```

Нека припомним:

[МодификаториЗаДостъп] (Access Modifiers): Определят откъде може да бъде извикан методът.

public: Методът е достъпен от всяко място. (Най-често срещаният)

private: Методът е достъпен само в рамките на класа, в който е дефиниран.

```
using System;

2 references
public static class GradeProcessor
{
    1 reference ДОСТЪПЕН И ВИДИМ НАВСЯКЪДЕ
    public static void GradeName(double grade)
    {
        string gradeName = GetGrade(grade);
        Console.WriteLine($"Оценка: {gradeName} ({grade}) ");
    }

    ДОСТЪПЕН САМО ВЪТРЕ В КЛАСА, НЕВИДИМ ЗА Program.cs
    1 reference
    private static string GetGrade(double grade)
    {
        return grade switch
        {
            >= 5.50 => "Отличен",
            >= 4.50 and <= 5.49 => "Много добър",
            >= 3.50 and <= 4.49 => "Добър",
            >= 2.99 and <= 3.49 => "Среген",
            _ => "Слаб"
        };
    }
}
```

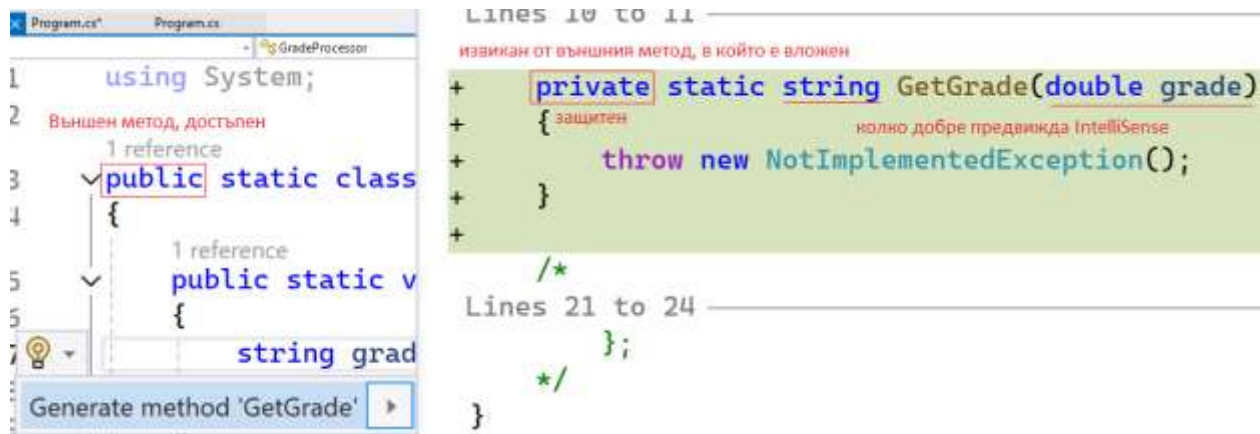
Когато започнем да пишем първия метод, виждаме грешка. Метод GradeName извиква в себе си за изпълнение вложен GetGrade, който още не е готов.

```
public static class GradeProcessor
{
    1 reference
    public static void GradeName(double grade)
    {
        string gradeName = GetGrade(grade);
        Console.WriteLine($"Оцен

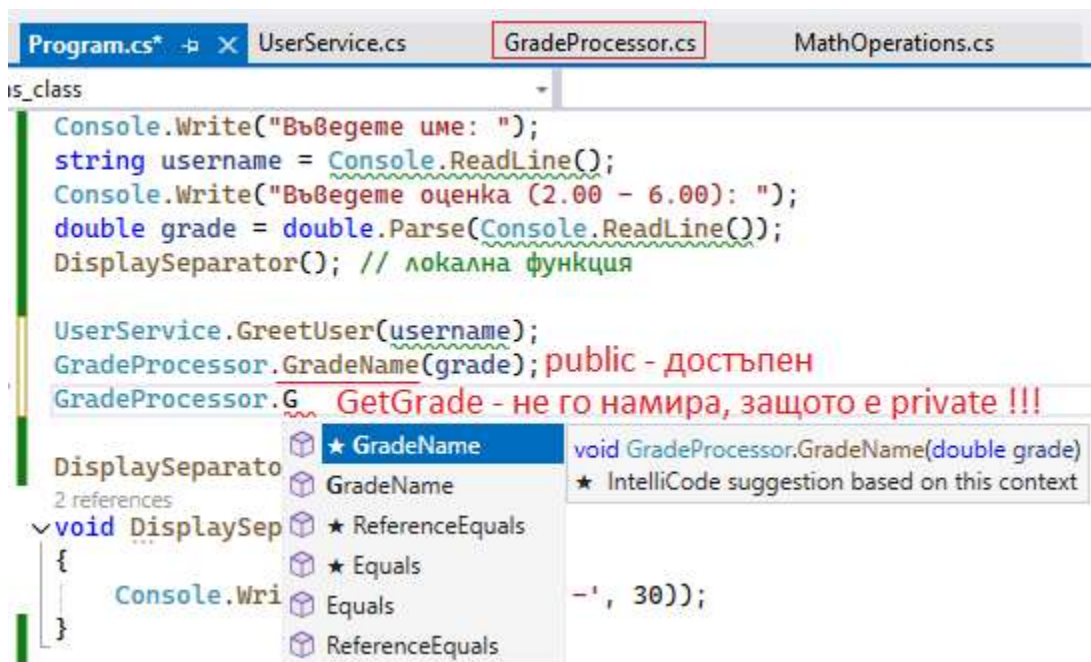
        CS0103: The name 'GetGrade'
        does not exist in the current
        context

        ЕДИН МЕТОД В ДРУГ МЕТОД (ВЛОЖЕН)

        Ctrl + . за автокорекция
    }
}
```



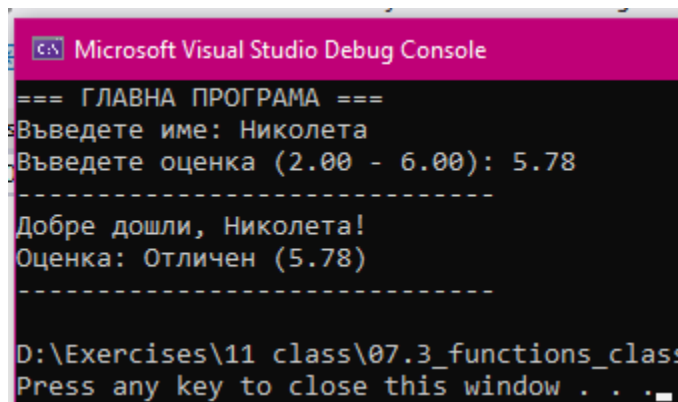
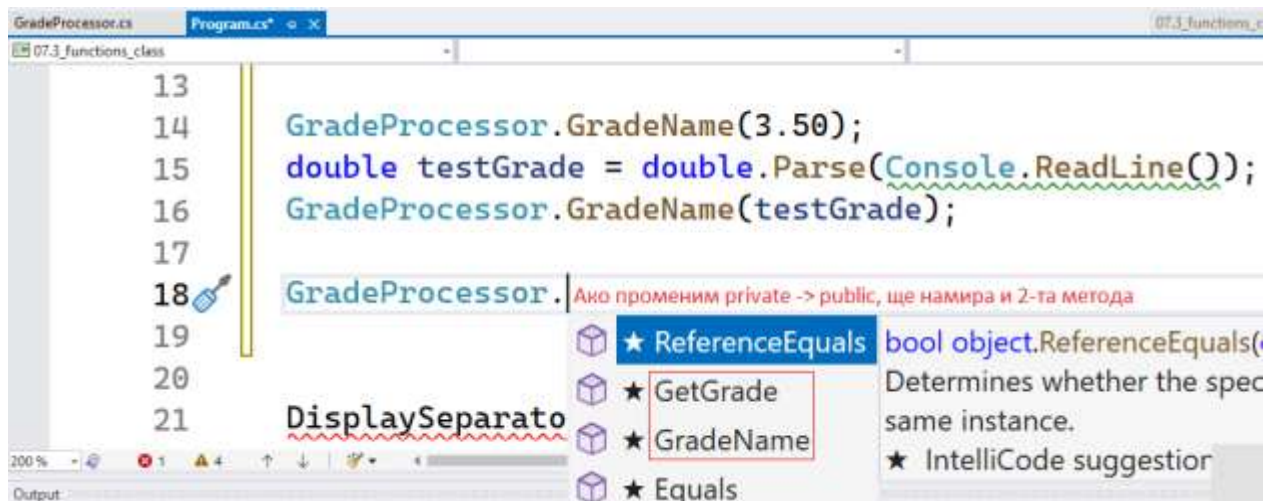
Т.е. по-добре е външния метод да бъде public, а вложения в него – private!



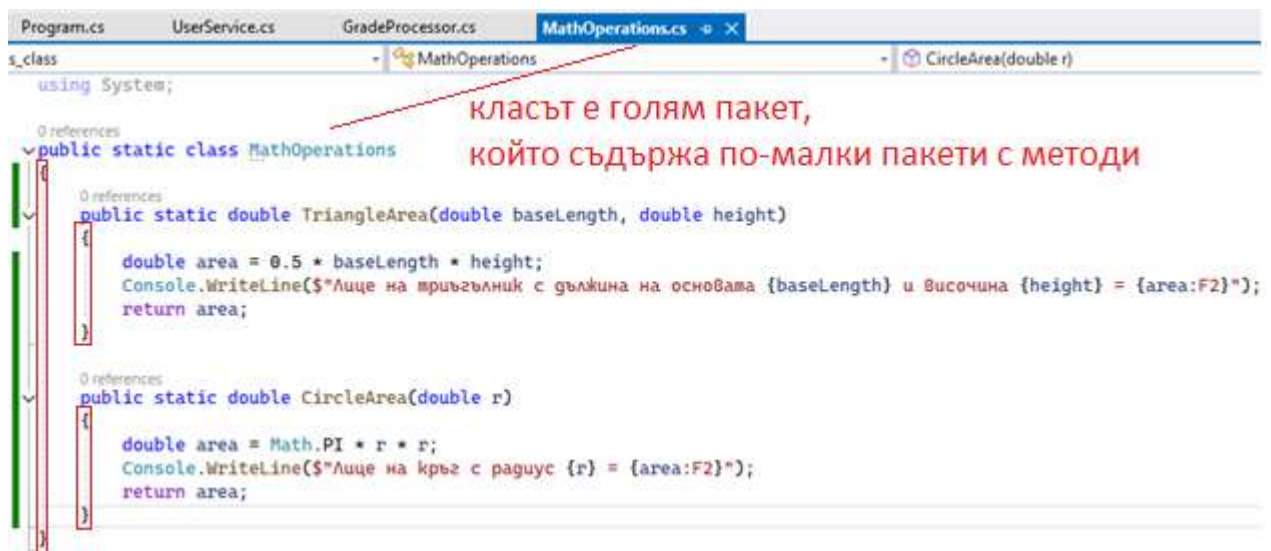
Т.е. виждаме, че public GradeName е достъпен от външен клас Program.cs, но GetGrade – не, защото е private – той е достъпен само в рамките на класа (файла), в който се намира. Това е защита. Ако променим вложения метод GetGrade да бъде public, ще го намери веднага и него, но пак повтарям, че по-добре е, когато един метод е вложен в друг, да не може да го достъпваме отвсякъде независимо, а да го оставим private!!!

* външен public – самостоятелен, общодостъпен

* вътрешен private - в тялото/кода на друг метод. Той е зависим и неравностоен спрямо метода, в който е вложен.



Последно да създадем 3-я файл за примерни математически операции, в които да видим 2 публично достъпни метода за лица на триъгълник и кръг:



Съответно може да се върнем в Program.cs и да ги извикаме

The screenshot shows the `Program.cs` file in Visual Studio. The code includes calls to `UserService.GreetUser`, `GradeProcessor.GradeName`, and `MathOperations.TriangleArea`. A red annotation highlights the call to `MathOperations.CircleArea` with the text "и двата метода за public, затова достъпни". An IntelliSense popup is visible for `MathOperations.CircleArea`, showing its signature: `double MathOperations.CircleArea(double r)`. The popup also lists other methods like `TriangleArea` and `CircleArea`.

```

Program.cs*  UserService.cs  GradeProcessor.cs  MathOperations.cs
15_class
Console.WriteLine("Въведете име: ");
string username = Console.ReadLine();
Console.WriteLine("Въведете оценка (2.00 - 6.00): ");
double grade = double.Parse(Console.ReadLine());
DisplaySeparator(); // локална функция

UserService.GreetUser(username);
GradeProcessor.GradeName(grade);

DisplaySeparator(); // локална функция

MathOperations.TriangleArea(3, 4);
MathOperations.CircleArea(7);

void DisplaySeparator()
{
    Console.WriteLine(new string('-', 30));
}

```

Нека тестваме с примерни стойности за променливите:

The screenshot shows the `Program.cs` file in Visual Studio. The code is the same as the previous screenshot, but with test values added: `MathOperations.TriangleArea(3, 4);` and `MathOperations.CircleArea(7);`. A red box highlights these two lines with the text "ДОСТЪПНИ, ТЕСТВАМЕ С ВХОДНИ СТОЙНОСТИ".

```

Program.cs*  UserService.cs  GradeProcessor.cs  M
15_class
Console.WriteLine("Въведете име: ");
string username = Console.ReadLine();
Console.WriteLine("Въведете оценка (2.00 - 6.00): ");
double grade = double.Parse(Console.ReadLine());
DisplaySeparator(); // локална функция

UserService.GreetUser(username);
GradeProcessor.GradeName(grade);

DisplaySeparator(); // локална функция
ДОСТЪПНИ, ТЕСТВАМЕ С ВХОДНИ СТОЙНОСТИ
MathOperations.TriangleArea(3, 4);
MathOperations.CircleArea(7);

DisplaySeparator(); // локална функция
3 references
void DisplaySeparator()
{
    Console.WriteLine(new string('-', 30));
}

```

```
Microsoft Visual Studio Debug Console

=== ГЛАВНА ПРОГРАМА ===
Въведете име: Николета
Въведете оценка (2.00 - 6.00): 5.97
-----
Добре дошли, Николета!
Оценка: Отличен (5.97)
-----
Лице на триъгълник с дължина на основата 3 и височина 4 = 6.00
Лице на кръг с радиус 7 = 153.94
-----
D:\Exercises\11 class\07.3_functions_class\bin\Debug\net8.0\07.
Press any key to close this window . . .
```